LfHL: Learning from a Handful of Labels to Predict AI Performance on Unseen Benchmarks

Arno Demarteau **Elias Hühne** **Quentin Geoffroy**
University of California, Berkeley
ademarteau@berkeley.edu

Abstract

The Predictive AI Evaluation Competition asks for the probability that an AI system answers a question correctly on a benchmark that was never seen in training, given between zero and 31 of that system's revealed answers. We describe our entry and what we learned while building it under a local evaluation that mirrors the platform: benchmark-level five-fold cross-validation on 1,720 system–benchmark pairs from 142 public benchmarks, scored with the competition's Brier area under the learning curve (ALC). Three findings shape the method. First, nothing computable before any label is revealed (text and format features, system metadata, tree-based interactions, difficulty judgements of a small language model) improves ALC on unseen benchmarks by more than 0.001. Second, once a system's own labels enter through a calibrated Bayesian update, three different prior shapes agree to within 0.0001 (ALC 0.2108–0.2109) and tag shifts and a text prior add only 0.0013 (0.2096): the information in a pair's own labels is essentially exhausted, while reacting naively to one label is worse than not learning at all. Third, the evaluation interface shares every revealed label of a run with all targets. Our model, LfHL (Learning from a Handful of Labels), is a hierarchical logistic model that pools a benchmark-level shift, tag shifts and per-question shifts across systems; it lowers ALC to 0.2067, 0.2026 and 0.1968 when 2, 5 or up to 20 systems share a benchmark, better in all five folds (95% interval of the largest gain -0.0150 to -0.0106). Uncertainty-based label acquisition does not help. Finally, we characterise the formative platform, disclose a one-parameter calibration of the starting probability to its unusually hard pairs, and show that identical uploads score with a standard deviation of 0.015.

1 Introduction

Running every AI system on every benchmark is expensive, and many evaluation decisions only need a good estimate of how a system will do. Item response theory [5, 6, 12] and its recent uses for efficient evaluation [10, 13] show that system ability and item difficulty can be estimated from a fraction of the responses *within* a benchmark. The Predictive AI Evaluation Competition (PAIEC) [15] asks a harder question: for a benchmark that no training data covers, predict $P(\text{correct})$ for each (system, question) from the question's text, the system's metadata, and $n \in \{0, 1, 3, 7, 15, 31\}$ of the system's own revealed answers on that benchmark.

This is a cold start at the level of the whole benchmark. Benchmarks differ in format, domain and even in what a correct label means, so what a model learns about one does not obviously carry over to the next. The question we organised our work around was therefore not "which model" but *where the usable information is*. Our answers, each backed by cross-validated experiments, are:

40th Conference on Neural Information Processing Systems (NeurIPS 2026). Workshop: Predictive AI Evaluation Competition (PAIEC).

- **A platform-faithful local evaluation** (Section 2): benchmark-level five-fold cross-validation over 1,720 pairs that reproduces the platform's split, nested budgets, shared `labeled` lists and streaming acquisition, with every learned parameter fitted on training folds only.
- **Budget-0 information does not transfer; a pair's own labels saturate quickly** (Sections 4 and 5): text, metadata, gradient-boosted and language-model features gain at most 0.001 ALC on unseen benchmarks, three prior shapes for the own-label update tie at ALC 0.2108–0.2109, and tag shifts and a text prior reach 0.2096.
- **LfHL: pooling labels across systems** (Section 3): a hierarchical Bayesian offset model that shares a benchmark level, tag shifts and per-question shifts among the systems of a run. With one system it reduces to the own-label model (up to repeated question texts), and with 2 to 20 systems it lowers ALC by 0.0029 to 0.0128.
- **An analysis of the formative platform** (Section 6): the hidden pairs are hard (mean accuracy 0.33), runs rarely contain two systems of one benchmark, single formative scores are noisy, and our calibrated model is within 0.0013 ALC of the best possible pair-level predictor there.

2 Task, data and evaluation protocol

Task. A *pair* is one system s on one benchmark b with at least 80 distinct questions. The platform splits a pair's questions once, at random, into an acquisition pool and an evaluation pool of equal size. Acquisition candidates stream past one at a time and an optional policy (`labeling.py`) decides which to reveal; without one, the platform samples uniformly. At each budget $n \in \{0, 1, 3, 7, 15, 31\}$ the model predicts every evaluation question given the revealed labels; budgets are nested. With BS_n the Brier score [2] at budget n, a pair's score is

$$\mathrm{ALC} = 0.1\,\mathrm{BS}_0 + 0.2\,(\mathrm{BS}_1 + \mathrm{BS}_3 + \mathrm{BS}_7 + \mathrm{BS}_{15}) + 0.1\,\mathrm{BS}_{31}, \tag{1}$$

the trapezoid rule on $\log_2(n+1)$, and the suite score is the mean over pairs with equal weight (constant 0.5 scores 0.25). Repeated trials of a question are separate targets.

The evaluation interface pools labels. The organizers' ingestion code[1] sends, at each budget checkpoint, *one* `labeled` list containing every label purchased so far in the run, and every target of that checkpoint is predicted with it. A target therefore sees the revealed answers of all other systems in the run, including those on the same benchmark, all at the same budget as its own and none at budget 0. Both official baselines filter `labeled` to the target system and so leave this information unused. Our main model (Section 3.4) uses it.

Data. We train only on the public Measurement DB [1] at a pinned revision (`54074453`). Of 281 benchmark directories, 163 have binary responses at question level; we exclude the three marked `release = withheld`, leaving 160 benchmarks with 12,985 scoreable pairs on 142 of them. Four facts guided the design (all measured on these data):

- *Systems rarely repeat*: 98% of system ids occur in one benchmark only; the metadata a model receives is sparse (`normalized_name` 48% filled, release date 33%, reasoning effort 0.1%), and a name-matched accuracy on another benchmark correlates only 0.23 with accuracy on this one.
- *Pairs differ a lot*: pair accuracy has mean 0.54 (0.50 when each benchmark counts at most 20 pairs, as in our evaluation) and standard deviation 0.27, a spread of 1.62 on the logit scale (1.29 around each benchmark's mean).
- *Question difficulty is shared across systems*: within a benchmark, the other systems' mean on the same question ranks a system's correct answers with AUC 0.87 (sample of 40 benchmarks).
- *Questions do not recur across benchmarks*: 0.1% of the 3.9 M public questions appear in another benchmark, and none outside the benchmark's own family.

In-sample ceilings on all scoreable pairs make the stakes concrete: predicting each benchmark's mean scores Brier 0.217, each pair's accuracy 0.174, each question's mean over systems 0.133. The pair level is reachable from a few labels; the question level only through other systems' answers.

[1] `tools/streaming_ingestion.py` of the official baseline repository (commit `cc88a95`), lines 147–181.

Local evaluation. `scripts/run_eval.py` assigns the 160 benchmarks to five folds (seed 0); each fold's predictors are fitted from scratch on the other four and scored on the held-out benchmarks, so every benchmark is scored once by a model that never saw it. Per benchmark we score the first 20 pairs by system id and at most 200 evaluation questions per pair, 1,720 pairs on 142 benchmarks in total. Each pair is split 50/50 with a seed derived from (benchmark, system) and reveals a seeded prefix of its acquisition order, as the platform does. To mirror pooling, the pairs of a benchmark are shuffled and cut into *pools* of M systems; every member is predicted with the pool's shared list (each member's own first n labels) at the same budget n. To test acquisition policies we simulate the streaming coordinator: between checkpoints the not-yet-revealed candidates stream past and the policy, called with the `labeling.py` signature and the current prediction, buys or skips. Because all runs score the same 1,720 pairs, we compare models with paired differences and report 95% intervals from a bootstrap over benchmarks (pairs of one benchmark are correlated), together with the number of folds in which the candidate is better. The 286 unit tests include equivalence checks, for example that the fast vectorised scorers used for fitting agree with the runtime predictions to 10^{-10}.

3 Method

All our models predict on the logit scale and differ in which shifts they learn from revealed labels. We build them up in the order in which we tested them; Table 1 reports each step.

3.1 Own labels: a Bayesian offset per pair

The simplest useful model gives every question of a new pair the same starting logit ℓ and learns one pair shift $\delta \sim \mathcal{N}(0, \tau^2)$:

$$\operatorname{logit} P(y_{sq} = 1) = \ell + \delta_{sb}. \tag{2}$$

After n revealed labels we approximate the posterior of δ by a Gaussian (Laplace approximation [16]: Newton steps to the mode, curvature for the variance v) and predict with the probit approximation of the logistic–Gaussian integral [7], $p = \sigma\big(m/\sqrt{1 + \pi v/8}\big)$, which keeps early predictions close to the base rate. The conjugate alternative is the shrunk mean $p = (\alpha p_0 + k)/(\alpha + n)$ for k correct of n, the posterior mean under a Beta prior [3]. Both are fitted by minimising the training ALC of simulated episodes, which replay the platform's split and reveal order on the training benchmarks (at most 20 pairs per benchmark). The fitted values match the data's own spread: $\tau = 1.58$ in every fold against a measured logit spread of 1.62, and $\alpha \approx 2.6$ against a method-of-moments estimate of 2.4.

3.2 Tag shifts within a benchmark

Many benchmarks label their questions with `key=value` tags (`task=qa3`, `lang=ja`). Their meaning is specific to the benchmark, so a tag weight learned on training benchmarks does not transfer; a tag *shift* learned from the pair's own revealed questions can. We add one shift $\gamma_g \sim \mathcal{N}(0, \sigma^2)$ per tag value g, choosing the grouping keys from the revealed questions alone (a key needs at least four revealed questions, two or more values, and repeating values; long or hash-like values are dropped). Evaluation questions with a tag value not yet revealed receive its prior variance σ^2 in the prediction.

3.3 A text prior for the starting logit

To give questions different starting points we fit an L2-regularised logistic regression [9] on hashed features [17] of what `predict` receives: 22 hand-made features (length, digits, code, mathematics, number of answer options and the implied guessing rate, release year, size and reasoning effort of the system) and hashed tokens of the question text, tags and system fields, in 2^{16} slots. Pairs are weighted equally, and the regularisation is chosen on held-out *benchmarks*. The prior enters as $a_{sq} = \ell + c + \lambda\,(f_{sq} - \mu)$, where f_{sq} is its logit and λ a learned trust. Because the prior was fitted on the training benchmarks, the logits used to fit $(c, \lambda, \tau, \sigma)$ are out-of-fold (three inner folds); $\lambda = 0$ recovers the tag model exactly, so the prior can only be used if it helps on held-out data.

3.4 LfHL: pooling across systems

For system s answering question q of benchmark b, our final model, LfHL, is

$$\operatorname{logit} P(y_{sq} = 1) = a_{sq} + \beta_b + u_s + \sum_{g \in G(q)} \gamma_g + v_q, \tag{3}$$

$$\beta_b \sim \mathcal{N}(0, \rho\,\tau^2), \qquad u_s \sim \mathcal{N}\big(0, (1-\rho)\,\tau^2\big), \qquad \gamma_g \sim \mathcal{N}(0, \sigma^2), \qquad v_q \sim \mathcal{N}(0, \omega^2).$$

The benchmark level β_b and the tag shifts are shared by every system in `labeled` on that benchmark; u_s belongs to one system; v_q is a per-question shift, informative only for questions some system has revealed. The share ρ splits the pair-shift variance between a level everyone learns from and an individual part. With a single system, $\beta_b + u_s \sim \mathcal{N}(0, \tau^2)$, so the model is *exactly* the own-label model of Sections 3.1–3.3 (tested to 10^{-9}), except that a system's own revealed answer now also moves evaluation questions with identical visible text (below); with more systems, one system's answers move the others' predictions through the level, the tags and the shared questions.

Inference. For each benchmark in `labeled` we compute one joint Laplace posterior over (β_b, u, γ) with Newton's method, then the question shifts v_q one-dimensionally with the other coefficients fixed. A target's prediction sums the posterior means of its coefficients, takes as variance their posterior covariance plus the prior variance of the parts no label informs yet (the system's own shift if it has no labels, unseen tag values), and applies the probit approximation; a question that no system has revealed has no shift v_q. Questions are matched by their *visible* content (text and tags), exactly as at runtime: many benchmarks reuse one prompt for questions that differ only in an attached file the model never sees (one benchmark has 1,200 questions but 600 distinct texts), and tuning had to see those collisions too.

Fitting. ℓ and τ come from a grid search of the own-label offset (31 levels $\times$ 41 values of τ), (c, λ) and then (τ, σ) from small grids with out-of-fold prior logits, and finally $\rho \in \{0, \ldots, 0.95\}$ and $\omega \in \{0, \ldots, 3\}$ on training benchmarks whose pairs are cut into pools of five. The fitted values are stable: $\tau = 1.58$, $\sigma = 0.75$ (0.5 in one fold), $\lambda \in [0.5, 1]$, $\rho = 0.8$ and $\omega = 1.5$ in every fold. Trained on all 160 benchmarks (675 s on a laptop CPU), the submitted parameters are $\lambda = 0.75$, $\tau = 1.58$, $\sigma = 0.75$, $\rho = 0.8$, $\omega = 1.5$.

Runtime. The submission is pure `numpy`: `model.py`, four runtime modules and the fitted parameters in a 0.5 MB archive, with no pretrained model and no network access. Since all targets of a checkpoint share one `labeled` list, the posterior of each benchmark is computed once per list and cached; item and system parts of the prior logit are cached separately.

4 Results

Table 1 and Figure 1 show the progression.

Reacting to labels must be calibrated. The organizers' empirical-mean baseline predicts the observed accuracy of the revealed labels, which after one label is 0 or 1: its Brier score at budget 1 is 0.357, far worse than not learning at all (0.25). Any Bayesian update fixes this: the shrunk mean is better on 75% of pairs and in every fold (-0.0327).

The pair's own labels are used up quickly. The Beta prior of the shrunk mean, the logit-normal prior of the Bayesian offset, and a nonparametric kernel-density prior over pair accuracy (Section 5) all score 0.2108–0.2109. When three different prior shapes tie to the fourth decimal, the remaining error is not in the update rule but in the information it receives.

Item-level information helps only inside a benchmark. Tag shifts are the first item-level information that transfers to unseen benchmarks (-0.0008, five of five folds, from budget 7 on): they work because they are learned from the benchmark's own revealed questions instead of as global weights. The text prior adds -0.0005 and is the first model below 0.25 at budget 0, but its interval includes zero.

Table 1: Five-fold cross-validated Brier score by budget and ALC on 1,720 pairs of 142 public benchmarks (lower is better). Δ: paired difference in ALC to the row above (for $M = 2, 5, \leq 20$: to $M = 1$), with a 95% bootstrap interval over benchmarks; folds: number of the five folds in which the row is better. M: number of systems of one benchmark whose labels are shared. The oracle row knows each pair's true accuracy on its evaluation questions.

Model	BS_0	BS_1	BS_3	BS_7	BS_{15}	BS_{31}	ALC	Δ ALC [95% CI]	folds
Constant 0.5	.2500	.2500	.2500	.2500	.2500	.2500	.2500		
Empirical mean (organizers)	.2500	.3574	.2412	.2066	.1941	.1875	.2436	$-.0064$ [$-.0200$, $+.0059$]	4
Shrunk mean	.2507	.2301	.2134	.1998	.1923	.1871	.2109	$-.0327$ [$-.0372$, $-.0279$]	5
Bayesian offset	.2505	.2299	.2133	.1999	.1924	.1871	.2109	$-.0000$ [$-.0002$, $+.0002$]	3
+ tag shifts	.2505	.2299	.2133	.1991	.1908	.1842	.2101	$-.0008$ [$-.0012$, $-.0004$]	5
+ text prior	.2494	.2290	.2127	.1988	.1907	.1842	.2096	$-.0005$ [$-.0017$, $+.0007$]	4
LfHL, $M = 1$	.2494	.2290	.2127	.1988	.1907	.1842	.2096	$\lvert\Delta\rvert < .0001$ (text twins, §3.4)	–
LfHL, $M = 2$	.2494	.2241	.2082	.1962	.1890	.1827	.2067	$-.0029$ [$-.0038$, $-.0020$]	5
LfHL, $M = 5$	.2494	.2151	.2036	.1930	.1865	.1799	.2026	$-.0070$ [$-.0088$, $-.0054$]	5
LfHL, $M \leq 20$	.2494	.2055	.1971	.1886	.1815	.1734	**.1968**	$-.0128$ [$-.0150$, $-.0106$]	5
Oracle pair accuracy	.1800	.1800	.1800	.1800	.1800	.1800	.1800		

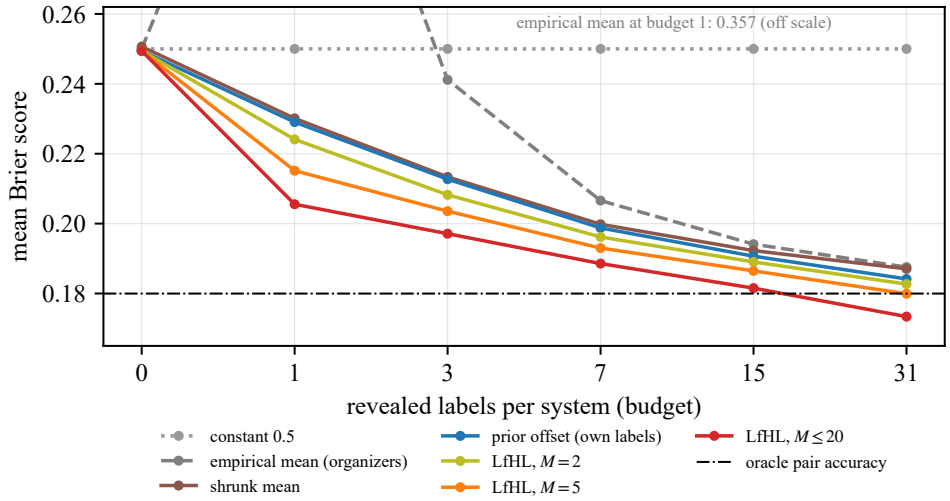


Figure 1: Mean Brier score by budget (same runs as Table 1). The naive empirical mean over-reacts to the first label (0.357 at budget 1, off scale). The Bayesian own-label models move smoothly; pooling labels across systems lowers every curve from budget 1 on, and with up to 20 systems it ends below the pair-accuracy oracle, which no own-label model can do.

Pooling is the largest lever. Sharing labels with one other system of the same benchmark gains 0.0029; with five, 0.0070; with up to 20, 0.0128, better in all five folds and on 78% of pairs. The gain is largest at budget 1 (from 0.2290 to 0.2055 with $M \leq 20$), where a single own label says little but the other systems' labels pin down the benchmark's level, and it persists at budget 31 (0.1842 to 0.1734), where shared questions and tags carry question-level information. At budget 31 LfHL beats the oracle that knows each pair's true accuracy (0.1800), which is impossible for any model that only learns one number per pair. With one system the model reproduces the own-label results fold for fold (text twins change 148 of 1,720 pairs slightly, the mean by less than 10^{-4}), so pooling costs nothing when `labeled` contains no other system.

5 What did not help, and why

Budget-0 features do not transfer to unseen benchmarks. On its own, the logistic text prior is worse than predicting 0.5 (Table 2), and the regularisation chosen on held-out benchmarks is the

Table 2: Ablations and negative results under the same protocol (ALC, 1,720 pairs). Δ against the reference in brackets, 95% bootstrap interval over benchmarks, and folds where the row is better.

Variant	ALC	Δ ALC [95% CI] (reference)	folds
Logistic text prior alone, no update	.2514	$+.0014$ [$-.0060$, $+.0089$] (constant 0.5)	3
Nonparametric prior over pair accuracy	.2108	$-.0001$ [$-.0003$, $+.0001$] (shrunk mean)	3
LightGBM prior instead of logistic	.2100	$+.0004$ [$-.0004$, $+.0011$] (+ text prior)	1
Streamed random acquisition	.2099	$+.0003$ [$-.0010$, $+.0016$] (fixed random order)	1
Uncertainty acquisition, $\lvert p-0.5\rvert \leq 0.10$	.2103	$+.0004$ [$-.0007$, $+.0015$] (streamed random)	2
Uncertainty acquisition, $\lvert p-0.5\rvert \leq 0.15$	.2107	$+.0008$ [$-.0004$, $+.0019$] (streamed random)	2

strongest in our grid in four of five folds. A diagnostic explains why: trained with one intercept per pair, so that only differences between questions are learned, the same features rank questions within a pair with AUC 0.66 on new questions of *known* benchmarks but 0.52 on unseen ones. Surface features largely act as a fingerprint of the benchmark (87% of the variance of question length is explained by the benchmark's identity): length, wording and tags vary between benchmarks rather than within them, and the meaning of a label varies too (a correct label is a refusal on some safety benchmarks). Gradient-boosted trees [4] on the same features, which can represent system $\times$ question interactions, were trusted less by the fit (the prior scale was set to zero in two folds) and were worse than the logistic prior in four of five folds.

A small language model's difficulty judgement captures 2% of the room. On 5,843 text questions from 40 benchmarks we asked Qwen2.5-1.5B- and 3B-Instruct [11] to rate difficulty and recorded their answer-token entropy and question likelihood (Appendix C). The 3B model's rating correlates with a question's facility in the expected direction in 82% of benchmarks (Spearman -0.14), but in leave-one-benchmark-out evaluation with the benchmark's true level given, it lowers the expected Brier score of a random system only from 0.1945 to 0.1932, against 0.1302 for a question oracle. The gain did not grow from 1.5B to 3B parameters, and the ratings did not predict benchmark levels either.

Uncertainty sampling does not beat random picks. Revealing the candidates the model rates closest to 0.5, the official baseline's acquisition rule and a classic active-learning heuristic [14], is slightly worse than random sampling at both widths, better in only two folds (Table 2). With one system, what matters is the pair's level, and any representative sample of labels estimates it equally well; preferring questions that the weak text prior rates near 0.5 makes the revealed sample less representative of the evaluation questions. Our simulation lets skipped candidates return at the next checkpoint, whereas the platform's stream drops them, so the platform version of the policy is more constrained and the conclusion holds a fortiori. We therefore submit no `labeling.py`.

6 The formative platform

During development the platform scores each upload on a fresh draw of about nine pairs from a hidden pool and reports per pair the Brier score and expected calibration error (ECE, 10 bins [8]) at every budget. We use this feedback for three things: to characterise the pool, to calibrate one parameter, and to judge how much a single score can tell.

Recovering the pool. If a model predicts the same probability c for all evaluation questions of a pair with accuracy a, then $\mathrm{BS} = (c-a)^2 + a(1-a)$ and $\mathrm{ECE} = |c-a|$, so $\mathrm{BS} - \mathrm{ECE}^2 = a(1-a)$, and a known $c \neq 0.5$ also fixes on which side of 0.5 a lies. Applying this to the published per-pair feedback of 52 formative runs of a model whose budget-0 prediction is constant per pair,[2] the identity holds across budgets (median standard deviation 0.003), the implied constant matches the known one, and repeated draws of a pair agree on the side of 0.5 in 101 of 101 cases. The pool has 7 benchmarks and 183 pairs with 44 to 113 evaluation questions each; within one draw a benchmark has one pair (274 times) or two (90 times), never more, so pooling across systems is rarely active on the formative

[2]These scoring logs were posted publicly by another participant together with their code; we used only the per-pair Brier and ECE values and no part of their method. Our own uploads are consistent with the estimates (Appendix D).

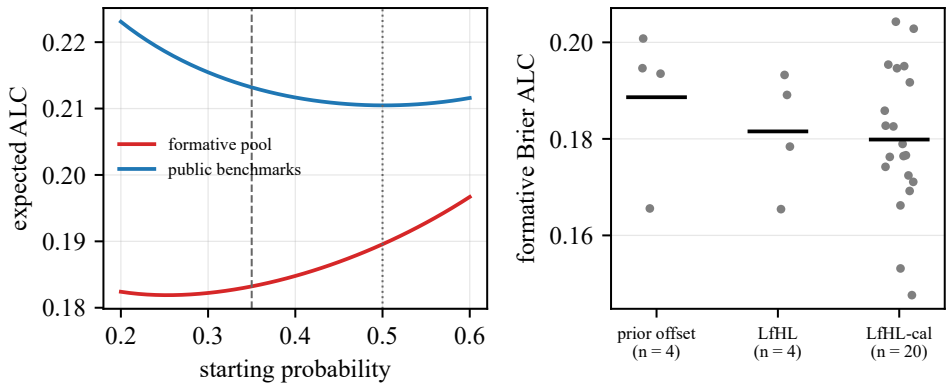


Figure 2: *Left:* expected ALC of the pair-level Bayesian offset ($\tau = 1.58$) as a function of the starting probability, on the formative pool's pair draws and on the public pairs (weighted as in our evaluation). Dotted: 0.5, the value learned from public data; dashed: 0.35, the calibrated start. *Right:* formative scores of our own uploads; each point is one upload of an unchanged archive, bars are means.

platform. Most importantly, the pairs are hard: mean accuracy 0.33 over draws, with 74% of draws below 0.5, against 0.50 on public pairs.

Calibrating the starting probability. With pairs this hard, our text prior pushed budget-0 predictions the wrong way on the platform (mean budget-0 Brier 0.258 over our first 44 pair draws, above 0.25 in 29), and a start of 0.5 is far from optimal there (Figure 2, left): over the pool's draws a start of 0.35 lowers the expected ALC of the pair-level offset from 0.1896 to 0.1832, while on public pairs it raises it from 0.2105 to 0.2132. The public data prefer 0.5, so this shift is not justified by training data; it is a calibration to the formative pool. We therefore keep both versions: the *trained* LfHL of Section 3, fitted on public data only, and a *calibrated* variant, LfHL-cal, that sets $\ell + c = \mathrm{logit}(0.35)$ and $\lambda = 0$ (text prior off), leaving every other parameter unchanged (`scripts/calibrate_submission.py`). Its budget-0 prediction is 0.392 for every question (the prior uncertainty pulls 0.35 towards 0.5). On the formative pool it is close to the best a pair-level model can do: the best constant start (0.25) gives 0.1819 and a Bayes predictor that knows the pool's accuracy distribution (leave one pair out) gives 0.1822, against an oracle of 0.1545 that knows each pair's accuracy.

How much a single formative score tells. Figure 2 (right) shows every upload of three unchanged archives. Twenty uploads of LfHL-cal score 0.180 on average with a standard deviation of 0.015, between 0.148 and 0.204; four uploads each of the trained LfHL and of the own-label prior offset average 0.182 and 0.189. Resolving a difference of 0.005 between two models at the 95% level would take about 70 uploads of each, and the best of many uploads mostly reflects the draw. Better evidence comes from pairs that recur across uploads: the platform's split and default acquisition are deterministic per pair, so a recurring pair is scored on the same questions with the same revealed labels. On the three pairs that recur in the scoring logs we have of both versions, all with low accuracy, LfHL-cal lowers the pair's ALC by 0.018 to 0.022.

Selected submission. We select LfHL-cal (Codabench submission #1020464) for the final evaluation, on the assumption that the common hidden test set resembles the formative pool; if the test pairs instead resemble the public ones, its expected cost is about 0.003 ALC (Figure 2, left). Both archives are rebuilt from the repository by the commands in Appendix A.

7 Limitations

- **Local protocol.** The pooled results assume a fixed number M of systems per benchmark sharing labels; the number in the final test is unknown and, on the formative platform, mostly one or two. The platform's candidate order is not published, so acquisition is simulated. Our folds

group benchmarks, not benchmark families (e.g. the ten `medhelm_*` benchmarks); since our gains come from labels at test time rather than from learned features this should matter little, but family-grouped folds would be stricter. For speed we score at most 20 pairs per benchmark and 200 questions per pair, with a single seed for splits and folds.

- **Calibration to the formative pool.** The calibrated start is fitted to the formative pool, whose accuracies we estimated from scoring feedback. If the final test differs, the trained LfHL is the better choice; we report both.
- **Question-level information.** The question oracle (0.133) is far below everything we reach. Pooling recovers part of it when several systems share a benchmark; we found no transferable source for a single system, but we did not test large language models or sentence embeddings at test time.
- **Scope of the pilot studies.** The language-model study covers 40 benchmarks and two model sizes; the within-pair diagnostics of the text prior one fold.

8 Conclusion

On unseen benchmarks, nearly all usable information arrives with revealed labels. A calibrated Bayesian update of one shift per pair extracts what a system's own labels hold, and three prior shapes confirm that little more is left there. The one large lever is that the platform shares labels across systems: LfHL, a hierarchical model that pools a benchmark level, tag shifts and per-question shifts, lowers ALC by up to 0.013 in cross-validation and degrades gracefully to the own-label model when no other system is present. We hope the protocol, the negative results and the analysis of formative noise are as useful to others as the model.

Reproducibility. Code, tests and the scripts that produce every number and figure are at `https://github.com/eliashuehne/paiec` (branch `pooling`); Appendix A lists the commands, run times and seeds.

Acknowledgments and Disclosure of Funding

We thank the PAIEC organizers for the competition, the data and the baselines. Coding assistants were used throughout; see Appendix E. This work received no external funding.

References

[1] AIMS Foundations. measurement-db: Item-level responses of AI systems on public benchmarks. Hugging Face dataset `aims-foundations/measurement-db`, revision `54074453`, 2026.

[2] Glenn W. Brier. Verification of forecasts expressed in terms of probability. *Monthly Weather Review*, 78 (1):1–3, 1950.

[3] Bradley Efron and Carl Morris. Data analysis using Stein's estimator and its generalizations. *Journal of the American Statistical Association*, 70(350):311–319, 1975.

[4] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[5] John P. Lalor, Hao Wu, and Hong Yu. Building an evaluation scale using item response theory. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pages 648–657, 2016.

[6] Frederic M. Lord. *Applications of Item Response Theory to Practical Testing Problems*. Lawrence Erlbaum Associates, Hillsdale, NJ, 1980.

[7] David J. C. MacKay. The evidence framework applied to classification networks. *Neural Computation*, 4 (5):720–736, 1992.

[8] Mahdi Pakdaman Naeini, Gregory F. Cooper, and Milos Hauskrecht. Obtaining well calibrated probabilities using Bayesian binning. In *Proceedings of the Twenty-Ninth AAAI Conference on Artificial Intelligence*, pages 2901–2907, 2015.

[9] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[10] Felipe Maia Polo, Lucas Weber, Leshem Choshen, Yuekai Sun, Gongjun Xu, and Mikhail Yurochkin. tinyBenchmarks: Evaluating LLMs with fewer examples. In *Proceedings of the 41st International Conference on Machine Learning*, 2024.

[11] Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

[12] Georg Rasch. *Probabilistic Models for Some Intelligence and Attainment Tests*. Danish Institute for Educational Research, Copenhagen, 1960.

[13] Pedro Rodriguez, Joe Barrow, Alexander Miserlis Hoyle, John P. Lalor, Robin Jia, and Jordan Boyd-Graber. Evaluation examples are not equally informative: How should that change NLP leaderboards? In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics*, pages 4486–4503, 2021.

[14] Burr Settles. Active learning literature survey. Computer Sciences Technical Report 1648, University of Wisconsin–Madison, 2009.

[15] Stanford AIMS Lab. Predictive AI Evaluation Competition (PAIEC). NeurIPS 2026 Competition Track, `https://aims.stanford.edu/competition`, 2026.

[16] Luke Tierney and Joseph B. Kadane. Accurate approximations for posterior moments and marginal densities. *Journal of the American Statistical Association*, 81(393):82–86, 1986.

[17] Kilian Weinberger, Anirban Dasgupta, John Langford, Alex Smola, and Josh Attenberg. Feature hashing for large scale multitask learning. In *Proceedings of the 26th International Conference on Machine Learning*, pages 1113–1120, 2009.

A Reproducing the results

All commands run from the repository root with Python 3.10 (`numpy`, `pandas`, `pyarrow`; `scikit-learn`, `scipy` and `lightgbm` for training; `matplotlib` for the figures). The data download needs a Hugging Face account with access to the gated Measurement DB.

```
git clone https://github.com/eliashuehne/paiec && cd paiec
git checkout pooling && git submodule update --init  # baseline at cc88a95
python -m venv .venv && source .venv/bin/activate    # or .venv\Scripts\activate
pip install -e ".[dev]" matplotlib
hf auth login && python scripts/download_data.py     # revision 54074453, ~5 GB
python -m pytest                                     # 286 tests

# Table 1, Figure 1 (five folds, seed 0; 0.5-3 h per command on a laptop CPU)
python scripts/run_eval.py --predictors constant empirical_mean shrunk_mean \
    offset tag_offset
python scripts/run_eval.py --predictors prior_offset gbdt_offset logreg_prior
python scripts/run_eval.py --predictors pooled_offset --pool-sizes 1 2 5 all
# Table 2, acquisition rows
python scripts/run_eval.py --predictors prior_offset --acquisition-items 100 \
    --policies random uncertainty10 uncertainty15

# Submission archives (training on all 160 benchmarks: 675 s)
python scripts/train_submission.py      # artifacts/pooled_offset.npz
python scripts/build_submission.py      # dist/submission.zip, official checker
python scripts/calibrate_submission.py --start 0.35 --no-text-prior
python scripts/build_submission.py \
    --artifacts artifacts/pooled_offset_start035.npz \
    --zip dist/pooled_offset_start035.zip

# Numbers and figures of this report (set the run folders in RUNS first)
python report/make_figures.py [--formative FORMATIVE_DRAWS.csv]
```

In the code, LfHL is the predictor `pooled_offset` and LfHL-cal the archive `dist/pooled_offset_start035.zip`. Each run of `run_eval.py` writes per-pair scores to `runs/<timestamp>/pairs.csv`. From these, `report/make_figures.py` computes the tables, the bootstrap intervals (4,000 resamples of benchmarks, seed 0) and the figures, and writes `report/results/numbers.json`. The nonparametric prior of Table 2 is on branch `logreg` (commit `aa0f0b2`). We do not redistribute the per-pair formative estimates behind Figure 2 (left); they are derived as described in Section 6 and Appendix D. All experiments ran on one laptop (Intel Core Ultra 9 185H, 32 GB RAM); only the language-model pilot used its GPU (NVIDIA RTX 4060 Laptop).

B Implementation details

Episodes. Fitting replays the platform protocol on training benchmarks: for at most 20 randomly chosen pairs per benchmark (each benchmark contributes at most 20 pairs of weight), the pair's questions are split with the same seeded 50/50 rule, the first 31 acquisition questions are the revealed labels, and at most 200 evaluation questions are scored. Every candidate parameter value is scored by the exact ALC of these episodes, with vectorised implementations that the tests check against the runtime code.

Grids. Own-label offset: 31 levels in $[-1.5, 1.5] \times 41$ values of τ in $[0.1, 10]$ (log-spaced). Shrunk mean: 81 values of α in $[0.05, 200]$. Text prior: $c \in \{-0.2, 0, 0.2\} \times \lambda \in \{0, 0.25, 0.5, 0.75, 1, 1.5\}$ at $\sigma = 0.75$, then $\tau \times \{0.7, 0.85, 1\} \times \sigma \in \{0.5, 0.75, 1\}$. Pooling: $\rho \in \{0, 0.2, 0.35, 0.5, 0.65, 0.8, 0.9, 0.95\} \times \omega \in \{0, 0.5, 1, 1.5, 2, 3\}$ on pools of five. Logistic regularisation $C \in [0.003, 1]$ chosen on an inner 20% of training benchmarks ($C = 0.01$ in the submission).

Features. Item: log characters, words and lines; digit and non-ASCII share; code, mathematics, JSON, question mark, empty content; detected answer options and the implied chance rate; interactors and tags present; hashed unigrams and bigrams of the first 2,000 characters, tag keys and `key=value`

pairs, interactor words. System: release-year offset, reasoning-effort level, parameter count parsed from names, missing-value flags; hashed provider, harness and version, effort, name words and extra settings. Each hashed token has value $1/\sqrt{\text{count}}$.

Learned parameters per fold. $\ell = 0$ (fold 3: -0.1), $c = 0$, $\tau = 1.58$, $\sigma = 0.75$ (fold 3: 0.5), $\lambda = 0.5, 0.5, 1.0, 1.0, 0.5$, $\rho = 0.8$, $\omega = 1.5$; shrunk mean $p_0 \in [0.49, 0.51]$, $\alpha \approx 2.57$.

C Language-model difficulty pilot

We sampled 5,843 questions with plain text (no attached-file stubs) from 40 public benchmarks with at least eight systems each; the target is a question's facility, the share of systems answering it correctly. Qwen2.5-1.5B-Instruct and Qwen2.5-3B-Instruct ran in fp16 (3 forward passes per question, 36 minutes for 1.5B) and produced four signals: a self-rated difficulty from 1 to 9 (expected value over the digit tokens), the entropy and top probability of the first answer token, and the question's negative log-likelihood. Within-benchmark Spearman correlations with facility were -0.12 (1.5B, expected sign in 68% of benchmarks) and -0.14 (3B, 82%) for the rating, and below 0.1 in magnitude for the other signals. For leave-one-benchmark-out evaluation we fitted a linear map from the standardised signals to the centred logit facility on the other benchmarks (slope shrunk by half), and set the held-out benchmark's level so that the mean prediction equals its true mean facility, which isolates the ordering of questions. The expected Brier score of a random system fell from 0.1945 (level only) to 0.1930 (1.5B) and 0.1932 (3B), against 0.1302 for the per-question oracle: 2% of the available room. Using the mean rating to predict a benchmark's level was inconsistent (-0.003 with 1.5B, $+0.003$ with 3B, against a constant level). The models were used offline only and are not part of the submission.

D Formative-pool recovery

Each formative scoring log lists, per pair and budget, the number of evaluation questions, the Brier score and the ECE. For a constant prediction all questions fall in one ECE bin, so the identity of Section 6 holds exactly; we take $v = \text{BS} - \text{ECE}^2$ averaged over budgets and draws, solve $a(1-a) = v$, and keep the root whose implied constant $c = a \pm \text{ECE}$ at budget 0 is closest to the known prediction of the logged model. The recovered constant agrees with the known one (median exactly; 74% of draws within 0.03), and recurring pairs agree on the side of 0.5 in all 101 cases. Our own calibrated upload predicts 0.392 for every question at budget 0, so its budget-0 ECE equals $|0.392 - a|$ and the same identity applies to it. In its scoring log (8 pairs) the budget-0 Brier score is below 0.25 on seven pairs and above it only on a pair of the pool's one easy benchmark, as the recovered accuracies predict; its mean budget-0 Brier score is 0.204, against 0.258 over 44 pair draws of our earlier uploads with the text prior. Figure 2 (left) evaluates the closed-form expected ALC of the pair-level offset, $\sum_n w_n \sum_k \binom{n}{k} a^k (1-a)^{n-k} [p_{nk}^2 - 2p_{nk}a + a]$, where p_{nk} is the model's prediction after k of n correct, over the 454 recovered pair draws (draw-weighted) and the 12,985 public pairs (weighted as in our evaluation, at most 20 pairs per benchmark).

E Disclosures

Training data and models. The submission is trained only on the 160 public benchmarks of Measurement DB (revision `54074453`). It uses no pretrained model. Qwen2.5-1.5B/3B-Instruct were used offline in the pilot of Appendix C only. The starting probability of LfHL-cal was set from formative scoring feedback (Section 6), including publicly posted scoring logs of another participant's formative runs.

Other teams. During development we read code that other participants released publicly and scored two public submissions under our protocol for reference; some share ideas with ours, such as pooling labels across systems or a lower starting level for formative pairs. Our pooled model grew out of our own pooling simulation, run before we read that code, and all code in our repository is our own.

Coding assistants. We used Claude Code (Anthropic) and Codex (OpenAI) to write and refactor code and tests, to run and summarise experiments, and to draft this report. Every number in the report is produced by code in the repository, and the authors checked the experiments, the code and the text.

Contributions. A.D. designed and implemented the predictors, the fitting procedures, the pooled and acquisition evaluation, the submission and the formative analysis, and wrote the report. E.H. set up the repository and the original evaluation harness (metrics, split, pairs, evaluation) on which ours builds, and explored neural and gradient-boosted cold-start predictors. Q.G. built the feature-screening harness with grouped folds and sealed benchmarks, and the system-identity and release-date features. All authors discussed the results and edited the report.